

The Curious Case of Exploding DecPOMDPs: Containing the Fire through Policy Counting^{*}

Nazlı Nur Karabulut^[0000–0001–7958–5627] and Tanya Braun^[0000–0003–0282–4284]

Computer Science Department, University of Münster, Münster, Germany
`{nnur.karabulut,tanya.braun}@uni-muenster.de`

Abstract. Decentralised partially observable Markov decision processes (DecPOMDPs) provide a general framework for modelling multi-agent decision making under uncertainty. However, DecPOMDPs are known to suffer from exponential complexity in the number of agents. One way to combat this intractability in agent numbers is to look at partitions of agents that exhibit a form of symmetry among agents, allowing for a compact encoding by counting. However, a challenge arises as the policy space explodes, even though the model complexity and evaluation cost reduce to a polynomial dependence. In this paper, we redirect our focus from counting agents to counting policies, which actually enables tractability in agent numbers for so called policy-counted DecPOMDPs. Further, we present policy-counted dynamic programming using the compact representation to solve policy-counted DecPOMDPs efficiently.

1 Introduction

Large-scale agent systems involve a huge number of interacting agents that have to coordinate decisions in uncertain environments. To formalise the setting, decentralised partially observable Markov decision processes (DecPOMDPs) provide a framework to compute a joint policy for a set of cooperative agents that maximises some joint utility measure in an environment that is considered to be stationary and describable by a probabilistic transition model. When agents are self-interested, the problem is formalised as a partially observable stochastic game (POSG), in which each agent has its own utility function.

However, solving multi-agent decision making problems is notoriously difficult as the number of potential policies escalates exponentially with the number of agents. To deal with this problem, researchers have proposed various approaches: Early work focuses on dynamic programming using pruning techniques for POSGs [4]. Building on this basis, Szer et al. present point-based dynamic programming for DecPOMDPs [21] and Seuken et al. propose memory-bounded dynamic programming [20]. The dynamic programming operator was later extended with approximate pruning techniques to improve scalability for POSGs [12]. In addition to dynamic programming-based approaches, Szer et al.

^{*} Full version including appendix of a paper accepted at the 17th International Conference on Scalable Uncertainty Management (SUM2026) under the same name.

[22] propose multiagent A* (MAA*), a policy search algorithm that groups action–observation histories of agents at the same stage when they share the same optimal Q-values in a Bayesian game formulation of the problem [17]. Newer work optimises MAA* even further [10,11], working on the doubly-exponential dependence on the horizon among other things. Additional research has examined more structured subclasses to improve scalability, e.g., for POSGs, examples include zero-sum formulations [26,5,24], one-sided POSGs where only one agent is affected by uncertainty [7,6,2], and common-payoff POSGs that restrict agents to share identical rewards [3].

Another lane of work focuses on so-called lifting [18]. Lifting exploits symmetries between objects, also sometimes referred to as exchangeability, which can be used for tractable inference [15]. In multi-agent systems, lifting is applied to agents, allowing for a more compact and tractable problem formulation using partitions of agents, with the number of partitions being much smaller than the number of agents. The basic assumption, first described by Braun et al. [1], is that, within a partition, permutations of joint actions and observations have the same effect on the joint transition or reward, allowing for counting actions and observations in histograms. Adding in an assumption about conditional independence among agents of a partition even allows for working with a single representative agent per partition without counting necessary, leading to a drastic reduction in complexity for so-called isomorphic DecPOMDPs. However, without that assumption, a reduction can only be made in model complexity and evaluation cost, while —almost counter-intuitively— the policy space explodes [1]. More recently, we have presented isomorphic POSGs and lifted dynamic programming to work with the compact encoding, yielding a runtime polynomial in the number of agents [8]. We have extended this work to counting policies to circumvent an explosion in the policy space for partitioned POSGs [9].

Invigorated by the progress made in counting POSGs, this work takes on the case of exploding DecPOMDPs under counting, a challenge left open by Braun et al. [1] in addition to a dedicated solution method, by extending the counting focus to policies. Specifically, the contributions are threefold: (i) a new definition of counting DecPOMDPs that allows for counting policies, preventing the explosion in the policy space, (ii) an updated utility calculation including an analysis that shows tractability in agent numbers, and (iii) a dynamic programming operator to solve policy-counted DecPOMDPs. As such, we solve the above-mentioned challenge and additionally provide a dedicated solution method. When showing tractability, we assume a fixed number of partitions that is much smaller than the number of agents.

The paper is organised as follows: We start with DecPOMDPs and dynamic programming. Then we analyse the problem of exploding DecPOMDPs and show how to use counting for policies to circumvent the problem, followed by counting dynamic programming. We end with a conclusion. Due to space constraints, the proofs are relegated to the appendix.

2 Preliminaries

In this section, we define DecPOMDPs and provide an overview of dynamic programming for DecPOMDPs.

2.1 Decentralised POMDPs

The definition is based on [16] and [19], using random variables S with $\text{ran}(S)$ referring to the set of (range) values it can take. Bold symbols denote sets, (time) steps t are denoted by superscript t , sequences over a discrete interval $[t_s : t_e]$ are denoted by superscript $t_s : t_e$, and omitting an element i from a set or sequence is denoted by subscript $-i$. To add an element to a sequence, we use $\circ$.

Definition 1. A (ground) DecPOMDP M is a tuple $(\mathbf{I}, S, \mathbf{A}, T, R, \mathbf{O}, \Omega, \tau)$,

- $\mathbf{I}$ a set of N agents,
- S a random variable with a set of states as a range,
- $\mathbf{A} = \{A_i\}_{i \in \mathbf{I}}$ a set of decision random variables, each A_i with a set of local actions as range,
- $T(S', S, \mathbf{A}) = P(S' \mid S, \mathbf{A})$ a transition function,
- $R(S, \mathbf{A})$ a reward function,
- $\mathbf{O} = \{O_i\}_{i \in \mathbf{I}}$ a set of random variables, each O_i with a set of local observations as range,
- $\Omega(\mathbf{O}, S) = P(\mathbf{O} \mid S)$ a sensor function¹, and
- τ a horizon.

Each agent i follows a local policy π_i^t mapping local observation histories $o^{0:t}$ to actions a , which can be represented as a depth- t policy tree (see Fig. 1). A joint policy is a tuple $\boldsymbol{\pi} = (\pi_1, \dots, \pi_n)$. The semantics of the ground model M is given by the joint policy space Π_M . The DecPOMDP problem asks for the joint policy that yields the maximum expected utility (with horizon τ):

$$MEU(M) = (\arg \max_{\boldsymbol{\pi} \in \Pi_M} U_M(\boldsymbol{\pi}), \max_{\boldsymbol{\pi} \in \Pi_M} U_M(\boldsymbol{\pi})) \quad (1)$$

where $U_M(\boldsymbol{\pi})$ is calculated recursively over τ steps, by summing over joint observations and next states, and then weighted according to the prior $T(s^0, \cdot, \cdot)$, i.e., $U_M(\boldsymbol{\pi}) = \sum_{s^0 \in \text{ran}(S)} T(s^0, \cdot, \cdot) U_M^\pi(s^0, \perp^{0:0})$ with

$$U_M^\pi(s^t, \mathbf{o}^{0:t}) = R(s_t, \boldsymbol{\pi}(\mathbf{o}^{0:t})) + \sum_{s^{t+1} \in \text{ran}(S)} T(s^{t+1}, s^t, \boldsymbol{\pi}(\mathbf{o}^{0:t})) \sum_{\mathbf{o}^{t+1} \in \text{ran}(\mathbf{O})} \Omega(\mathbf{o}^{t+1}, s^{t+1}) U_M^\pi(s^{t+1}, \mathbf{o}^{0:t+1}) \quad (2)$$

¹ For ease of exposition regarding counting and to be consistent with [1], we omit $\mathbf{A}$ from Ω , which does not make the problem inherently easier to solve [19].

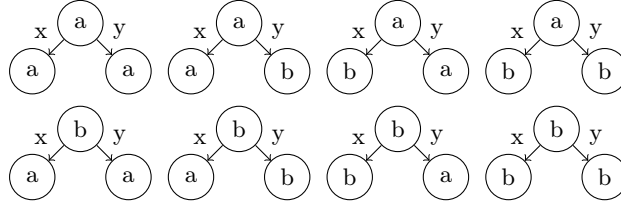


Fig. 1. Policies at level $t = 1$ available to agents with actions a, b and observations x, y

ending with $U_M^\pi(s, \mathbf{o}^{0:\tau-1}) = R(s, \pi(\mathbf{o}^{0:\tau-1}))$, where $\pi(\mathbf{o}^{0:t})$ denotes the joint action at step t ($\pi(\perp)$: root actions) and $\mathbf{o}^{0:t+1} = \mathbf{o}^{0:t} \circ \mathbf{o}^{t+1}$ the updated history.

In a DecPOMDP, all agents share a single joint reward function $R(S, \mathbf{A})$ and cooperate to maximize the total expected utility. However, each agent decides what to do next on its own, not being able to observe the full state or the other agents' observations. Nonetheless, agents are not fully independent since they depend on a joint state and receive the reward jointly.

2.2 Dynamic Programming

Dynamic programming is a solution method for multi-agent decision making that is shared between DecPOMDPs and POSGs [16]. It builds increasingly deeper policy trees from sub-trees for individual agents, while eliminating weakly-dominated policies at each depth [21]. Algorithm 1 shows the dynamic programming operator which consists of three steps. First, an exhaustive backup is performed to build all possible policies $\Pi_i^t = \{\pi_{i,j}^t\}_{j=1}^{m_t}$ of depth t from the input policies $\Pi_i^{t-1} = \{\pi_{i,j}^{t-1}\}_{j=1}^{m_{t-1}}$ for each agent i . Then, for each policy $\pi_{i,j}^t \in \Pi_i^t$, the operator calculates the corresponding value vector $V_{i,j}^t$, representing the value of that policy for each possible combination of state $s \in \text{ran}(S)$ and policies of the other agents $\pi_{-i}^t \in \Pi_{-i}^t$, which is defined as follows with $\pi = \pi_{i,j}^t \circ \pi_{-i}$

$$V_i(s, \pi) = R(s, \pi(\perp)) + \sum_{s' \in \text{ran}(S)} T(s', s, \pi(\perp)) \sum_{\mathbf{o} \in \text{ran}(\mathbf{O})} \Omega(\mathbf{o}, s) V_i(s', \pi \circ \mathbf{o}) \quad (3)$$

where $\pi \circ \mathbf{o}$ denotes the sub-policies after following an observation. During pruning, the operator eliminates policies that are weakly-dominated by others (solvable by a linear programme, see App. A.1), meaning that over the complete space of $\text{ran}(S) \times \Pi_{-i}^t$ there is always another policy with at least as high a value, i.e.,

$$\forall s \in \text{ran}(S), \pi_{-i}^t \in \Pi_{-i}^t \exists \pi_{i,j'}^t \in \Pi_i^t : V_i(s, \pi_{i,j'}^t \circ \pi_{-i}^t) \leq V_i(s, \pi_{i,j}^t \circ \pi_{-i}^t) \quad (4)$$

3 A Case of Exploding DecPOMDPs

The basic idea of lifting DecPOMDPs is that the agent set is partitioned into sets of agents that behave indistinguishably among each other, meaning that

Algorithm 1 Multi-agent Dynamic Programming Operator

```

function MA-DP(set of policies  $\Pi_i^{t-1}$  for each agent  $i \in \mathbf{I}$  with value vectors  $\mathbf{V}_i^{t-1}$ )
  for each agent  $i \in \mathbf{I}$  do
     $\Pi_i^t \leftarrow$  Perform exhaustive backup using  $\Pi_i^{t-1}$ 
     $\mathbf{V}_i^t \leftarrow$  Calculate new value vectors
  while  $\exists \pi_{i,j}^t \in \Pi_i^t$  : Eq. (4) holds for some  $i \in \mathbf{I}$  do
     $\Pi_i^t \leftarrow \Pi_i^t \setminus \{\pi_{i,j}^t\}$ ,  $\mathbf{V}_i^t \leftarrow \mathbf{V}_i^t \setminus \{v_{i,j}^t\}$ 
  return  $\{(\Pi_i^t, \mathbf{V}_i^t)\}_{i \in \mathbf{I}}$ 

```

it does not matter which agent performs which action within a partition, only how many do so [18,13,1]. In such a case, it is enough to count the agents and store the counts in a histogram. To illustrate the point, consider $N = 12$ indistinguishable agents with the actions a, b available as an example. Imagine that 8 agents perform a while the remaining 4 perform b . Then, there exist $\binom{12}{8} = \frac{12!}{8!4!} = 495$ ways to make the 12 agents perform the actions, all of which result in the same outcome ρ . Therefore, we can construct a histogram [4, 8] that maps to ρ , replacing 495 mappings. While this leads to a drastic decrease in model complexity, the policy space explodes in the formalisation chosen in [1]. To understand why, we next discuss the assumptions and design choices, left at times implicit in the original paper. Then, we look at the (in)tractability results.

3.1 Assumptions & Definitions

Braun et al. [1] name as their basic assumption indistinguishability among agents. They posit that indistinguishability requires agents to have the same local actions and observations, and that another consequence is that permutations of actions (observations) map to the same outcome in transition, reward, and sensor function, which we posit boils down to two assumptions, which we formalise next.

The first assumption is that all agents i, j in a partition $\mathcal{J} \subseteq \mathbf{I}$ have the same action and observation space:

$$\text{ran}(A_i) = \text{ran}(A_j) \wedge \text{ran}(O_i) = \text{ran}(O_j) \quad (5)$$

which follows the requirement definition in [1]. The second assumption is that the transition, reward, and sensor functions T , R , and Ω are symmetric, meaning that exchanging actions or observations between agents of a partition has no effect on the outcome of T , R , and Ω , which can be formally expressed for the actions of two agents as:

$$\begin{aligned} \forall \mathbf{a} \in \text{ran}(\mathbf{A}), \mathbf{a} = (a_i \circ a_j \circ \mathbf{a}_{-i,-j}) : \\ T(s', s, a_i \circ a_j \circ \mathbf{a}_{-i,-j}) = T(s', s, a_j \circ a_i \circ \mathbf{a}_{-i,-j}) \end{aligned} \quad (6)$$

The same holds for the actions in R and the observations in Ω . We now generalise Eq. (6) to entire partitions, to build interchangeability among agents within

a partition. The set of agents $\mathbf{I}$ is divided into K partitions $\mathcal{I}_1, \dots, \mathcal{I}_K$ of indistinguishable agents. For each partition $\mathcal{I}_k$, let $\mathbf{a}_k = (a_i)_{i \in \mathcal{I}_k}$ and $\mathbf{o}_k = (o_i)_{i \in \mathcal{I}_k}$ denote the joint action and observation tuples of the agents in that partition. With the extension of symmetric behaviour to partitions, swapping the actions or observations of agents within the same partition does not change a function's outcome. Formally, for any permutation θ of $\mathcal{I}_k$,

$$T(s', s, \mathbf{a}_k, \mathbf{a}_{-k}) = T(s', s, \theta(\mathbf{a}_k), \mathbf{a}_{-k} \mathbf{o}_{-k}, s). \quad (7)$$

with the same holding for any permutations of actions in T and observations in Ω . These assumptions are analogous to those made for counting in POSGs [9].

Equations (5) and (7) allow for encoding actions and observations using histograms, which can then be used to compactly encode T , R , and Ω by replacing the individual mappings from all $\theta(\mathbf{a}_k)$ to an outcome ρ with a single mapping $h \mapsto \rho$, with h being a histogram of the counts of actions in $\mathbf{a}_k$, which is identical for all $\theta(\mathbf{a}_k)$. The same holds for the permutations of observations in $\mathbf{o}_k$. Formally, histograms are defined as follows:

Definition 2. *Given a set of n agents $\mathbf{I}$ and a set of m values $\nu = \{v_1, \dots, v_m\}$, a histogram h is defined as $h = \{(v_l, n_l)\}_{l=1}^m$ with $\sum_{l=1}^m n_l = n$ where each n_l denotes the number of instances with value v_l . We use $h(v_l)$ to refer to n_l . $[n_1, \dots, n_m]$ is used as a shorthand. A counting random variable (CRV) $\#_{\mathbf{I}}[V]$ is a syntactic construct whose range consists of all histograms over the range values of variable V , i.e., $\nu = \text{ran}(V)$, that meet $\sum_l n_l = |\mathbf{I}|$.*

Considering the shorthand example $h = [4, 8]$ from above for actions a, b and $n = 12$, the full version reads $h = \{(a, 8), (b, 4)\}$.

Braun et al. [1] then define a partitioned DecPOMDP that uses such histograms as range values for actions and observations in each partition for every affected component of the DecPOMDP.

Definition 3 ([1]). *A counting DecPOMDP $\bar{M}_c$ is a tuple $(\bar{\mathbf{I}}, \bar{S}, \bar{\mathbf{A}}_c, \bar{T}, \bar{R}, \bar{\mathbf{O}}_c, \bar{\Omega}, \tau)$, with*

- $\bar{\mathbf{I}}$ a partitioning $\{\mathcal{I}_k\}_{k=1}^K$ of N agents, with $n_k = |\mathcal{I}_k|$ and $|\bar{\mathbf{I}}| = \sum_k n_k = N$,
- $\bar{S}$ a random variable with a set of states as range,
- $\bar{\mathbf{A}}_c = \{\#_{\mathcal{I}_k}[A_k]\}_{k=1}^K$ a set of decision CRVs,
- $\bar{T}(S', S, \bar{\mathbf{A}}_c)$ a transition function with counted actions,
- $\bar{R}(S, \bar{\mathbf{A}}_c)$ a reward function with counted actions,
- $\bar{\mathbf{O}}_c = \{\#_{\mathcal{I}_k}[O_k]\}_{k=1}^K$ a set of CRVs,
- $\bar{\Omega}(\bar{\mathbf{O}}_c, S)$ a sensor function with counted observations, and
- τ a horizon.

The equivalence between a DecPOMDP fulfilling Eqs. (5) and (7) and a counting DecPOMDP can be shown by construction, replacing all mappings with permutations of inputs with a single mapping with a histogram and vice versa (see App. B.1).

3.2 (In)Tractability Results

For a ground DecPOMDP, the model complexity lies in $O(s^2 a^N)$ for T , $O(sa^N)$ for R , and $O(so^N)$ for Ω with $s = |\text{ran}(S)|$, $a = \max_{i \in \mathcal{I}} |\text{ran}(A_i)|$, and $o = \max_{i \in \mathcal{I}} |\text{ran}(O_i)|$, while the cost of evaluating a joint policy lies in $O(so^{N^\tau})$ and the policy space size in $O(a^{\frac{N(o^\tau - 1)}{o - 1}})$ [16]. Using CRVs allows for reducing the model size and evaluation cost from exponential to polynomial.

Theorem 1 ([1], Thm. 2). *A counting DecPOMDP $\bar{M}_c$ allows for representation [i.e., model] and cost to depend polynomially on N .*

The theorem holds by the range sizes of the CRVs, which are capped by N^a , $a = \max_k |\text{ran}(A_k)|$, and N^o , $o = \max_k |\text{ran}(O_k)|$, as an upper bound on the number of histograms given by the binomial coefficient $\binom{N+v-1}{v-1} \leq N^v$, $v \in \{a, o\}$ [13]. Compared to the ground case, a and o are replaced by N^a and N^o , while K replaces N in the exponent, yielding a polynomial dependence on N , assuming that $K \ll N$.

However, the number of policies explodes: Since the size of the policy space contains o in the exponent, replacing o with N^o means that N remains in the exponent, all the while a is replaced with N^a , leading to an expression that has N in the base as well as the exponent.

Corollary 1. *The policy space of a counting DecPOMDP $\bar{M}_c$ depends exponentially on N .*

The crux is the choice of CRVs in the action and observation set used for building policies. In contrast, we use plain variables and count policies instead, which is a small change with profound effects.

4 Containing the Fire by Counting Policies

For a polynomial dependence on the agent numbers, we keep plain variables in the action and observation sets, enabled by Eq. (5), but use CRVs in the transition, reward, and sensor function, enabled by Eq. (7). With plain variables, there is a set of (representative) policies available in each partition, which allows for structuring the policy space by counting the number of agents following each representative policy and updating the *MEU* calculations. We first formalise counted policies before renewing the definition of counting DecPOMDPs. Last, we look at the effect of counted policies on the *MEU* calculation and complexity.

4.1 Counted Policies

Let us consider the example from the beginning of Section 3 with 12 agents and actions a, b available. At step $t = 1$, assuming two possible observations x, y , each of the 12 agents has the same 8 policies available, see Fig. 1. Instead of tracking all possible policies for each individual agent, we can use these 8 policies as representative policies to keep track of agent behaviour in histograms: For

example, the partition policy in which all agents follow the first policy is encoded by $[12, 0, 0, 0, 0, 0, 0]$, while the other configuration in which one agent follows the second policy and the remaining agents follow the first one is encoded by $[11, 1, 0, 0, 0, 0, 0]$. In this way, all possible partition policies can be compactly encoded.

More formally, according to Eq. (5), the set of policies for any two agents i, j within a partition $\mathfrak{I}_k$ is established using the same sets of actions $A_i = A_j = A_k$ and observations $O_i = O_j = O_k$. As a result, their local policy space is the same, i.e., $\Pi_i = \Pi_j = \Pi_k$. These policies are referred to as *representative policies* Π_k of partition $\mathfrak{I}_k$ and can be counted for each partition.

Definition 4. Let the set of J representative policies of a partition $\mathfrak{I}_k$ be given by $\Pi_k = \{\pi_1, \dots, \pi_J\}$. A counted partition policy over Π_k is a histogram $h_k^\Pi = \{(\pi_j, m_j)\}_{j=1}^J$, where $m_j \geq 0$ is the number of agents in $\mathfrak{I}_k$ following policy π_j , and $\sum_{j=1}^J m_j = n_k$. The counted policy space for partition $\mathfrak{I}_k$, denoted by H_k^Π , is given by the set of all histograms meeting $\sum_{j=1}^J m_j = n_k$. The joint counted policy space over all partitions is given by $\mathbf{H}^\Pi = \times_{k=1}^K H_k^\Pi$. The counted policy space of all partitions except k is denoted by $\mathbf{H}_{-k}^\Pi$ and of all agents except some agent $i \in \mathfrak{I}_k$ by $\mathbf{H}_{-i}^\Pi$ where $\sum_{j=1}^J m_j = n_k - 1$ for H_k^Π without i .

4.2 A Renewed Definition

To distinguish the two definitions, we use *policy-counted* DecPOMDP as a name here, which is still a DecPOMDP with a partitioned agent set fulfilling Eqs. (5) and (7).

Definition 5. A policy-counted DecPOMDP $\bar{M}$ is a tuple $(\bar{\mathbf{I}}, \bar{S}, \bar{\mathbf{A}}, \bar{T}, \bar{R}, \bar{\mathbf{O}}, \bar{\Omega}, \tau)$, with

- $\bar{\mathbf{I}}$ a partitioning $\{\mathfrak{I}_k\}_{k=1}^K$ of N agents, $n_k = |\mathfrak{I}_k|$ and $|\bar{\mathbf{I}}| = \sum_k n_k = N$,
- $\bar{S}$ a random variable with a set of states as range,
- $\bar{\mathbf{A}} = \{A_k\}_{k=1}^K$ a set of decision random variables,
- $\bar{T}(S', S, \bar{\mathbf{A}}_c)$ a transition function with counted actions,
- $\bar{R}(S, \bar{\mathbf{A}}_c)$ a reward function with counted actions,
- $\bar{\mathbf{O}} = \{O_k\}_{k=1}^K$ a set of random variables,
- $\bar{\Omega}(\bar{\mathbf{O}}_c, S)$ a sensor function with counted observations, and
- τ a horizon.

where $\bar{\mathbf{A}}_c = \{\#\mathfrak{I}_k[A_k]\}_{k=1}^K$ and $\bar{\mathbf{O}}_c = \{\#\mathfrak{I}_k[O_k]\}_{k=1}^K$.

The difference between Def. 3 and Def. 5 only lies in $\bar{\mathbf{A}}_c$ and $\bar{\mathbf{O}}_c$ versus $\bar{\mathbf{A}}$ and $\bar{\mathbf{O}}$, which are CRVs in the former and plain variables in the latter case, as the set of actions and observations available. Since Eq. (7) holds in both definitions, $\bar{T}$, $\bar{R}$, and $\bar{\Omega}$ are the same with CRVs as inputs for actions and observations. This encoding captures the symmetric behaviour of agents within each partition and helps to reduce the model complexity by operating over counted representations instead of enumerating all individual agents. Theorem 2 establishes the equivalence between the ground DecPOMDP and its counted version of Def. 5.

Theorem 2. *A ground DecPOMDP M that meets Eqs. (5) and (7) is equivalent to a policy-counted DecPOMDP $\bar{M}$.*

The full proof can be found in App. B.2, which is based on the fact that one can be converted into the other by construction and vice versa. Having established the equivalence between ground and counting DecPOMDPs, we now show how the counting formulation affects the model complexity.

Theorem 3. *For a fixed number of partitions K , fixed action and observation spaces, and a fixed horizon τ , the model complexity of a counting DecPOMDP $\bar{M}_c$ is polynomial in the number of agents N .*

The full proof can be found in App. B.3, where we show that using histograms within agent partitions keeps the model size polynomial in N .

4.3 Counted Utility Calculation

As the policy space is given by joint counted policies, we can update the semantics and *MEU* calculation. The semantics of the policy-counted DecPOMDP $\bar{M}$ is given by the joint counted policy space $\mathbf{H}^{\Pi}$. The policy-counted DecPOMDP problem asks for the joint counted policy that yields the maximum expected utility (with horizon τ), which is again computed by recursively calculating the expected utility over the next states and observations, until reaching τ .

However, the sum over observations can now be turned into a sum over counted observations: Given a joint counted policy, each partition is partitioned again into groups following a representative policy according to the counted partition policy, with group sizes corresponding to the histogram counts. For a given (representative) observation history, the recursive call to the next utility as well as the remaining sub-policy are the same for these agents, making them indistinguishable among themselves. As such, for each of these groups, we can count observations in histograms over the (sub)group with a particular representative policy and observation history, and let the sum go over the cross product of these histograms. Conditioning these groups on their shared observation history means that groups split up further, which can in the worst case lead to singletons if the group size is small to begin with or τ is very large but the overall complexity is still going to be only polynomial as we show afterwards. Additionally, we only need this deep recursion for the utility definition here and are able to define policy-counted dynamic programming using a single step.

To formalise this setting, we define the space of representative observation histories as well as the space of observation histograms given a representative observation history.

Definition 6. *Given a step t , a representative observation history $p_k^{0:t,o}$ of a partition $\mathfrak{I}_k$ is a sequence of observations $o^{t'} \in \text{ran}(O_k)$ of length t , $t' \in \{1, \dots, t\}$, with $o^0 = \perp$ as an empty observation. Given a representative observation history $p_k^{0:t,o}$, let $p_k^{t',o}$ refer to the entry at point t' in the sequence and $p_k^{0:t-1,o}$ refer to the subsequence without the last entry $p_k^{t,o}$. Finally, let $P_k^{0:t,o}$*

denote the space of representative observation histories of length t for the agents in partition $\mathfrak{I}_k$, i.e., $P_k^{0:t,o} = \perp \times \times_{t'=1}^t \text{ran}(O_k)$.

Definition 7. Given a step t , a counted partition policy h_k^Π , and a representative policy $\pi \in \Pi_k$ for a partition $\mathfrak{I}_k$, a counted partition observation is a set of histograms $\{h_{k,p_k}^{o,\pi}\}_{p_k^{0:t,o} \in P_k^{0:t,o}}$ where

$$h_{k,p_k}^{o,\pi} = \{(o_l, n_l)\}_{o_l \in \text{ran}(O_k)}$$

with

$$\sum_{o_l \in \text{ran}(O_k)} n_l = \begin{cases} h_{k,p_k}^{o,\pi} (p_k^t) & \text{if } t > 0 \\ h_k^\Pi(\pi) & \text{if } t = 0 \end{cases} \quad (8)$$

and $p_k^{0:t,o} = p_k^{0:t-1,o} \circ p_k^t$. Let $H_k^{o,\pi}$ denote the space of counted partition observations for a given representative policy π , i.e., the set of all possible sets of histograms that fulfil Eq. (8). Given a joint counted policy $\mathbf{h}^\Pi$, let $H_k^{o,\Pi}$ denote the space of counted partition observations over all representative policies Π_k , i.e., $H_k^{o,\Pi} = \times_{\pi \in \Pi_k} H_k^{o,\pi}$, and $\mathbf{H}^{o,\Pi}$ the space of joint counted observations, i.e., $\mathbf{H}^{o,\Pi} = \times_{k=1}^K H_k^{o,\Pi}$.

Note that the observation histogram in Eq. (8) holds counts n_l for $p^{0:t,o} \circ o_l$. Based on these definitions, we can now rewrite Eq. (1) as follows:

$$MEU(\bar{M}) = (\arg \max_{\mathbf{h}^\Pi \in \mathbf{H}^\Pi} U_{\bar{M}}(\mathbf{h}^\Pi), \max_{\mathbf{h}^\Pi \in \mathbf{H}^\Pi} U_{\bar{M}}(\mathbf{h}^\Pi)) \quad (9)$$

where $U_{\bar{M}}(\mathbf{h}^\Pi)$ is calculated recursively over τ steps and weighted according to the prior $T(s^0, \cdot, \cdot)$, i.e., $U_{\bar{M}}(\mathbf{h}^\Pi) = \sum_{s^0 \in \text{ran}(S)} T(s^0, \cdot, \cdot) U_{\bar{M}}^{\mathbf{h}^\Pi}(s^0, \perp^{0:0})$ with

$$U_{\bar{M}}^{\mathbf{h}^\Pi}(s^t, (\mathbf{h}^{o,\Pi})^{0:t}) = R(s^t, \mathbf{h}^a) + \sum_{s^{t+1} \in \text{ran}(S)} T(s^{t+1}, s^t, \mathbf{h}^a) \sum_{\mathbf{h}^{o,\Pi} \in \mathbf{H}^{o,\Pi}} \text{Mul}(\mathbf{h}^{o,\Pi}) \cdot \Omega(\mathbf{h}^o, s^{t+1}) U_{\bar{M}}^{\mathbf{h}^\Pi}(s^{t+1}, (\mathbf{h}^{o,\Pi})^{0:t} \circ \mathbf{h}^{o,\Pi}) \quad (10)$$

with $\text{Mul}(\mathbf{h}^{o,\Pi})$ the multinomial coefficient, denoting the number of ground permutations encoded by the histograms [23], where for each partition $\mathfrak{I}_k$, the counted partition action $h_k^a \in \mathbf{h}^a$ as input to R and T is given by adding up, for each action $a \in \text{ran}(A_k)$, how often a is carried out over all representative policies and all representative observation histories using the counts in the latest counted partition observation if the observation history plus the newest observation map to a in the current representative policy, i.e.,

$$h_k^a = \{(a_l, n_l)\}_{a_l \in \text{ran}(A_k)} \quad \text{with} \quad n_l = \begin{cases} \sum_{\pi \in \Pi_k} \sum_{p \in P_k^{0:t-1,o}} \sum_{o \in \text{ran}(O_k)} h_{k,p}^{o,\Pi,t}(o) \mathbb{1}_{\pi_k(p \circ o) = a_l} & \text{if } t > 0 \\ \sum_{\pi \in \Pi_k} h_k^\Pi(\pi) \mid \pi(\perp) = a_l & \text{if } t = 0 \end{cases} \quad (11)$$

with $h_{k,p}^{o,\Pi,t} \in \mathbf{h}_k^{o,\Pi,t} \in (\mathbf{h}_k^{o,\Pi})^{o:t}$ and $\mathbf{1}$ being the indicator function returning 1 if $\pi_k(p \circ o) = a_l$. The counted observation $h_k^o \in \text{ran}(\#\mathcal{I}_k[O_k])$ as input to Ω is given by adding up, for each observation $o \in \text{ran}(O_k)$, how often o is observed over all representative policies and all representative observation histories in the latest counted partition observation, i.e.,

$$h_k^o = \{(o_l, n_l)\}_{o_l \in \text{ran}(O_k)} \quad \text{with} \quad n_l = \sum_{\pi \in \Pi_k} \sum_{p \in P_k^{0:t-1,o}} h_{k,p}^{o,\Pi,t}(o_l) \quad (12)$$

The utility calculation for a ground DecPOMDP and an equivalent policy-counted DecPOMDP is equivalent again, leading to the same maximum expected utility. The trick is that calculations that lead to the same result are only performed once in the counted case, which is possible due to the histograms essentially encoding when calculations lead to the same result.

Theorem 4. *The maximum expected utility for a ground DecPOMDP M that meets Eqs. (5) and (7) is equal to the maximum expected utility of the corresponding policy-counted DecPOMDP $\bar{M}$. That is, $MEU(M) = MEU(\bar{M})$.*

The full proof can be found in App. B.4, where we show that both models give the same utility by mapping ground policies to counted policies. Next, we look at evaluation cost and policy space size.

Theorem 5. *For a fixed number of partitions K , fixed representative action and observation spaces, and a fixed horizon τ , the cost of evaluating a joint counted policy and the size of the overall joint counted policy space in a policy-counted DecPOMDP depend polynomially on the number of agents N .*

The full proof can be found in App. B.5, which relies on the joint counted observation space and the joint counted policy space depending polynomially on the number of agents N , as $n_k \leq N$ agents are distributed onto the different representative observations and policies, which can be characterised using the binomial coefficient. Thus, we have reduced the complexity from an exponential one to a polynomial one for N . As such, a policy-counted DecPOMDP is tractable in the number of agents.

Corollary 2. *Given a fixed number of partitions K , fixed representative action and observation spaces, and a fixed horizon τ , a policy-counted DecPOMDP is tractable in the number of agents N .*

With the number of agents N no longer appearing in the exponent, we are able to solve larger problem instances in terms of N compared to the ground case, especially if the number of partitions K is small. Next, we use the counting encoding to speed up dynamic programming for DecPOMDPs.

5 Policy-Counted Dynamic Programming

In this section, we extend the standard dynamic programming operator for policy-counted DecPOMDPs. Because of the equivalence between the ground

and policy-counted versions of DecPOMDPs, we can show that the value computations are equivalent, allowing for computing values at a partition level as well as pruning representative policies.

The policy-counted dynamic programming operator still follows the three general steps of exhaustive backup, value computation, and pruning but on a partition-level. That is, Alg. 1 still applies but computes value vectors and performs pruning according to counted versions of those equations for each partition (see Eqs. (13) and (14)). During the exhaustive backup, the operator builds all *representative* policies of depth t based on the policies of depth $t - 1$ for each partition and then computes their corresponding value vectors. Since the operator builds increasingly deeper policies, the value computation stays at the top level, building its counted joint action from the root actions in the representative policies, i.e., with $t = 0$ in Eq. (11) to build $\mathbf{h}^a$, and uses empty representative histories, i.e., Eq. (8) with $t = 0$ as well to build $\mathbf{H}^{o, \Pi}$. Thus, for a state s and joint counted policy $\mathbf{h}_{-i}^{\Pi}$ of the other agents and partitions, the value computation for a given representative policy π_k is defined as follows with $\pi_k \oplus \mathbf{h}_{-i}^{\Pi}$ adding 1 to the corresponding count in h_k^{Π} of $\mathbf{h}_{-i}^{\Pi}$ to form a full joint counted policy:

$$\begin{aligned} \bar{V}_k(s, \mathbf{h}^{\Pi}) = & R(s, \mathbf{h}^a) + \sum_{s' \in \text{ran}(S)} T(s', s, \mathbf{h}^a) \\ & \sum_{\mathbf{h}^{o, \Pi} \in \mathbf{H}^{o, \Pi}} \text{Mul}(\mathbf{h}^{o, \Pi}) \cdot \Omega(\mathbf{h}^o, s') \bar{V}_k(s', \mathbf{h}^{\Pi} \cdot \mathbf{h}^{o, \Pi}) \end{aligned} \quad (13)$$

where $\mathbf{h} \cdot \mathbf{h}^{o, \Pi}$ denotes the policy histogram that follows from the observation histogram $\mathbf{h}^{o, \Pi}$. Specifically, given depth t of the representative policies Π_k^t used for $\mathbf{h}^{\Pi}$, the joint counted policy $\mathbf{h}^{\Pi}$, and the current joint counted observation $\mathbf{h}^{o, \Pi}$, the new policy histogram is calculated by adding up how often a subpolicy $\pi_l \in \Pi_k^{t-1}$ remains over the different representative policies $\pi \in \Pi_k^t$ when following the different observations $o \in \text{ran}(O_k)$ given the counts in $h_k^{o, \Pi} \in \mathbf{h}^{o, \Pi}$ for o , i.e.,

$$\mathbf{h} \cdot \mathbf{h}^{o, \Pi} = \{(\pi_l, n_l)\}_{\pi_l \in \Pi_k^{t-1}} \quad \text{with} \quad n_l = \sum_{\pi \in \Pi_k^t} \sum_{o \in \text{ran}(O_k)} h_k^{o, \Pi}(o) \mid \pi \cdot o = \pi_l$$

During pruning, the operator eliminates weakly dominated representative policies π_k based on the following equation (solvable by a linear programme, see App. A.2):

$$\text{for all } s \in \text{ran}(S), \mathbf{h}_{-i}^t \in \mathbf{H}_{-i}^t \exists \pi_{l \neq k}^t \in \Pi_k^t : \bar{V}_k(s, \pi_k^t \oplus \mathbf{h}_{-i}^t) \leq \bar{V}_k(s, \pi_l^t \oplus \mathbf{h}_{-i}^t) \quad (14)$$

Equation (14) ensures that, for each state and each counted policy of the other partitions, a representative policy is only pruned if there exists another one with equal or higher value.

Next, we show that counted dynamic programming using Eqs. (13) and (14) is equivalent to dynamic programming using Eqs. (3) and (4), which is a culmination of the formalisations and theorems so far.

Theorem 6. *Using policy-counted dynamic programming on a policy-counted DecPOMDP $\bar{M}$ is equivalent to using standard dynamic programming on a ground DecPOMDP M , in which Eqs. (5) and (7) hold.*

The full proof can be found in App. B.6, where we show that each step of the dynamic programming operator works equivalently in both the ground and counted models. Given the equivalence of the policy-counted and standard version of dynamic programming, correctness of the policy-counted version follows directly from the correctness of the standard version.

Corollary 3. *Policy-counted dynamic programming using Eqs. (13) and (14) is correct.*

Next, we show that policy-counted dynamic programming depends polynomially on the number of agents assuming a fixed and small number of representative actions and observations and partitions as well as horizon.

Theorem 7. *For a fixed number of partitions K , fixed representative action and observation spaces, and a fixed horizon τ , the runtime of policy-counted dynamic programming in a policy-counted DecPOMDP $\bar{M}$ depends polynomially on the number of agents N .*

The full proof can be found in App. B.7, which analyzes the polynomial runtime of the backup, value calculation, and pruning steps.

Appendix C shows the well-known DecTiger model [14] as ground, counting, and policy-counted DecPOMDPs to highlight the similarities and differences between the models. In summary, policy-counted dynamic programming is able to use the compact encoding of policy-counted DecPOMDPs to generate a solution that is equivalent to the ground solution in time polynomial in N instead of exponential in N , assuming all other parameters to be fixed. As the DecTiger example shows, the pay-off only arises if N is sufficiently large, i.e., if the assumption of $N \ll K$ indeed holds. Otherwise the counting of policies introduces too much overhead. Future work is left to deal with the remaining exponent that can still become rather large.

6 Conclusion

This paper presents policy-counted DecPOMDPs, which exploit a partitioned agent set under certain structural assumptions to reduce the complexity from exponential to polynomial in the number of agents, assuming all other parameters to be fixed. Specifically, we are able to use representative policies per partition, enabling counting how many agents follow which policy, which allows for a compact representation and considerably reduces the model’s size, computing cost,

and policy space, leading to tractability in agent numbers assuming a fixed number of partitions, horizon, and representative action and observation spaces. Additionally, we present a policy-counted dynamic programming for policy-counted DecPOMDPs that updates policies and values as well as prunes policies directly at the partition level. As a consequence, policy-counted DecPOMDPs offer a lot of potential for large-scale applications such as nanoscale medical systems.

In future research, we aim to further enhance scalability by working on policy encodings as well as applying lifting to large state spaces, to enable additional complexity reductions. Another idea is to look at approximations, considering that histograms are rarely needed in count steps of 1.

References

1. Braun, T., Gehrke, M., Lau, F., Möller, R.: Lifting in Multi-agent Systems under Uncertainty. In: UAI-22 Proc. of the 38th Conference on Uncertainty in Artificial Intelligence. pp. 1–8. AUAI Press (2022)
2. Carr, S., Jansen, N., Bharadwaj, S., Spaan, M.T.J., Topcu, U.: Safe policies for factored partially observable stochastic games. In: RSS-21 Proc. of Robotics: Science and Systems XVII. pp. 1–11. RSS Foundation (2021)
3. Emery-Montemerlo, R., Gordon, G., Schneider, J., Thrun, S.: Approximate solutions for partially observable stochastic games with common payoffs. In: AAAMAS-04 Proc. of the 3rd International Joint Conference on Autonomous Agents and Multiagent Systems. pp. 136–143. IEEE (2004)
4. Hansen, E.A., Bernstein, D.S., Zilberstein, S.: Dynamic programming for partially observable stochastic games. In: AAAI-04 Proc. of the 19th National Conference on Artificial Intelligence. vol. 4, pp. 709–715 (2004)
5. Horák, K., Bošanský, B.: Solving partially observable stochastic games with public observations. In: Proc. of the AAAI conference on Artificial Intelligence. pp. 547–552. AAAI Press (2019)
6. Horák, K., Bošanský, B., Kiekintveld, C., Kamhoua, C.: Compact Representation of Value Function in Partially Observable Stochastic Games. In: IJCAI-19 Proc. of the 28th International Joint Conference on Artificial Intelligence. pp. 350–356. IJCAI Organisation (2019)
7. Horák, K., Bošanský, B., Pěchouček, M.: Heuristic Search Value Iteration for One-sided Partially Observable Stochastic Games. In: AAAI-17 Proc. of the 31st AAAI Conference on Artificial Intelligence. pp. 558–564 (2017)
8. Karabulut, N.N., Braun, T.: Lifting partially observable stochastic games. In: International Conference on Scalable Uncertainty Management. pp. 201–216. Springer (2024)
9. Karabulut, N.N., Braun, T.: Counting agents in partially observable stochastic games. In: European Conference on Symbolic and Quantitative Approaches with Uncertainty. pp. 207–222. Springer (2025)
10. Koops, W., Jansen, N., Junges, S., Simao, T.D.: Recursive Small-step Multi-agent A* for Dec-POMDPs. In: IJCAI-23 Proceedings of the 32nd International Joint Conference on Artificial Intelligence. pp. 5402–5410. IJCAI Organization (2023)
11. Koops, W., Junges, S., Jansen, N.: Approximate Dec-POMDP Solving Using Multi-agent A*. In: IJCAI-24 Proceedings of the 33rd International Joint Conference on Artificial Intelligence. pp. 6743–6751. IJCAI Organization (2024)

12. Kumar, A., Zilberstein, S.: Dynamic programming approximations for partially observable stochastic games. In: FLAIRS-09 Proc. of the 22nd International Florida Artificial Intelligence Research Society Conference. AAAI Press (2009)
13. Milch, B., Zettelmoyer, L.S., Kersting, K., Haimes, M., Kaelbling, L.P.: Lifted Probabilistic Inference with Counting Formulas. In: AAAI-08 Proc. of the 23rd AAAI Conference on Artificial Intelligence. pp. 1062–1068. AAAI Press (2008)
14. Nair, R., Tambe, M., Yokoo, M., Pynadath, D.V., Marsella, S.: Taming Decentralized POMDPs: Towards Efficient Policy Computation for Multiagent Settings. In: IJCAI-03 Proc. of the 18th International Joint Conference on Artificial Intelligence. pp. 705–711. IJCAI Organization (2003)
15. Niepert, M., Van den Broeck, G.: Tractability through Exchangeability: A New Perspective on Efficient Probabilistic Inference. In: AAAI-14 Proc. of the 28th AAAI Conference on Artificial Intelligence. pp. 2467–2475. AAAI Press (2014)
16. Oliehoek, F.A., Amato, C.: A Concise Introduction to Decentralised POMDPs. Springer (2016)
17. Oliehoek, F.A., Whiteson, S., Spaan, M.T.: Lossless Clustering of Histories in Decentralized POMDPs. In: AAMAS-09 Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems. pp. 577–584. IFAAMAS (2009)
18. Poole, D.: First-order Probabilistic Inference. In: IJCAI-03 Proc. of the 18th International Joint Conference on Artificial Intelligence. pp. 985–991. IJCAI Organization (2003)
19. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. Pearson (2021)
20. Seuken, S., Zilberstein, S.: Memory-Bounded Dynamic Programming for DEC-POMDPs. In: IJCAI-07 Proceedings of the 21st International Joint Conference on Artificial Intelligence. pp. 2009–2015. IJCAI Organization (2007)
21. Szer, D., Charpillet, F.: Point-based dynamic programming for dec-pomdps. In: AAAI. vol. 6, pp. 1233–1238 (2006)
22. Szer, D., Charpillet, F., Zilberstein, S.: MAA*: A Heuristic Search Algorithm for Solving Decentralized POMDPs. In: UAI-05 Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence. pp. 576–583. ACM (2005)
23. Taghipour, N., Fierens, D., Davis, J., Blockeel, H.: Lifted Variable Elimination: Decoupling the Operators from the Constraint Language. *Journal of Artificial Intelligence Research* **47**(1), 393–439 (2013)
24. Tomášek, P., Horák, K., Aradhye, A., Bošanský, B., Chatterjee, K.: Solving partially observable stochastic shortest-path games. In: IJCAI-21 Proc. of the 30th International Joint Conference on Artificial Intelligence. pp. 4182–4189. IJCAI Organisation (2021)
25. Vaidya, P.M.: Speeding-up Linear Programming Using Fast Matrix Multiplication. In: 30th Annual Symposium on Foundations of Computer Science. pp. 332–337. IEEE Computer Society (1989)
26. Wiggers, A.J., Oliehoek, F.A., Roijers, D.M.: Structure in the value function of two-player zero-sum games of incomplete information. In: ECAI-16 Proc. of the 22nd European Conference on Artificial Intelligence. pp. 1628–1629. IOS Press (2016)

The Curious Case of Exploding DecPOMDPs: Containing the Fire through Policy Counting (Supplementary Material)

A Linear Programmes

This section provides the linear programmes used during the pruning step in dynamic programming, which repeats the pruning condition from the main paper and then lists the linear programme.

A.1 Ground DecPOMDP

Formally, a policy $\pi_{i,j}^t$ is pruned if the following holds,

$$\forall s \in \text{ran}(S), \pi_{-i}^t \in \Pi_{-i}^t \exists \pi_{i,j'}^t \in \Pi_i^t : V_i'(s, \pi_{i,j}^t \circ \pi_{-i}^t) \leq V_i'(s, \pi_{i,j'}^t \circ \pi_{-i}^t)$$

which can be solved by solving the following linear programme (prune $\pi_{i,j}^t$ if $d < 0$):

$$\begin{aligned} &\text{variables: } b_i(s, \pi_{-i}), d \\ &\text{maximise: } d \\ &\text{constraints: } \forall \pi_{i,j'}^t \in \Pi_i^t \\ &\quad \sum_{s \in \text{ran}(S)} \sum_{\pi_{-i} \in \Pi_{-i}^t} b_i(s, \pi_{-i}) V_i(s, \pi_{i,j}^t, \pi_{-i}^t) \\ &\quad - \sum_{s \in \text{ran}(S)} \sum_{\pi_{-i} \in \Pi_{-i}^t} b_i(s, \pi_{-i}) V_i(s, \pi_{i,j'}^t, \pi_{-i}^t) - d \leq 0 \\ &\quad \sum_{s \in \text{ran}(S)} \sum_{\pi_{-i} \in \Pi_{-i}^t} b_i(s, \pi_{-i}) = 1, b_i(s, \pi_{-i}) > 0 \end{aligned}$$

A.2 Policy-Counted DecPOMDP

Formally, a representative policy π_k^t is pruned if the following holds, which can still be solved using a linear program:

$$\forall s \in \text{ran}(S), \mathbf{h}_{-i}^t \in \mathbf{H}_{-i}^t \exists \pi_{l \neq k}^t \in \Pi_k^t : \bar{V}_k'(s, \pi_k^t \oplus \mathbf{h}_{-i}^t) \leq \bar{V}_k'(s, \pi_l^t \oplus \mathbf{h}_{-i}^t),$$

which can be solved by solving the following linear programme (prune $\pi_{k,l}^t$ if $d < 0$):

$$\text{variables: } b_k(s, \mathbf{h}_{-i}), d$$

$$\begin{aligned}
& \text{maximise: } d \\
& \text{constraints: } \forall \pi_{k,l'}^t \in \Pi_k^t \\
& \sum_{s \in \text{ran}(S)} \sum_{\mathbf{h}_{-i} \in \mathbf{H}_{-i}^t} b_k(s, \mathbf{h}_{-i}) V_k(s, \pi_{k,l}^t \oplus \mathbf{h}_{-i}^t) \\
& - \sum_{s \in \text{ran}(S)} \sum_{\mathbf{h}_{-i} \in \mathbf{H}_{-i}^t} b_k(s, \mathbf{h}_{-i}) V_k(s, \pi_{k,l'}^t \oplus \mathbf{h}_{-i}^t) - d \leq 0 \\
& \sum_{s \in \text{ran}(S)} \sum_{\mathbf{h}_{-i} \in \mathbf{H}_{-i}^t} b_k(s, \mathbf{h}_{-i}) = 1, b_k(s, \mathbf{h}_{-i}) > 0
\end{aligned}$$

B Full Proofs

B.1 Equivalence between a Ground DecPOMDP and a Counting DecPOMDP

This theorem shows equivalence between a ground DecPOMDP and a counting DecPOMDP based on Eqs. (5) and (7).

Theorem 1. *A ground DecPOMDP M that meets Eqs. (5) and (7) is equivalent to a counting DecPOMDP $\bar{M}_c$.*

Proof. To convert a DecPOMDP $M = (\mathbf{I}, S, \mathbf{A}, \mathbf{O}, T, R, \Omega, \tau)$ into a counting DecPOMDP $\bar{M}_c = (\bar{\mathbf{I}}, S, \bar{\mathbf{A}}_c, \bar{\mathbf{O}}_c, \bar{T}, \bar{R}, \bar{\Omega}, \tau)$, we apply Eqs. (5) and (7). First, agents within the same partition share the same action and observation spaces, meaning that for any $i, j \in \mathcal{J}_k$, $A_i = A_j$ and $O_i = O_j$, which is provided by Eq. (5). For each partition $\mathcal{J}_k$, we introduce the CRVs $\#_{\mathcal{J}_k}[A_k]$ and $\#_{\mathcal{J}_k}[O_k]$ for actions and observations, whose ranges are all histograms over the local action space $\text{ran}(A_k)$ and observation space $\text{ran}(O_k)$. To build the transition, sensor, and reward functions, based on Eq. (7), we replace $(A_i)_{i \in \mathcal{J}_k}$ and $(O_i)_{i \in \mathcal{J}_k}$ with CRVs $\#_{\mathcal{J}_k}[A_k]$ and $\#_{\mathcal{J}_k}[O_k]$ in the inputs of the transition, reward, and sensor function for each partition, with a histogram h_k mapping to ρ for all permutations of a partition input $\mathbf{a}_k$ or $\mathbf{o}_k$ mapping to ρ for which h is the histogram representation.

To convert a counting DecPOMDP into a ground DecPOMDP fulfilling Eqs. (5) and (7), we essentially reverse the steps above: The agent set $\mathbf{I}$ is the union of the partitions $\bigcup_{k=1}^K \mathcal{J}_k$. The action and observation sets are given by $\{A_{k,i}\}_{i \in \mathcal{J}_k}$ and $\{O_{k,i}\}_{i \in \mathcal{J}_k}$, fulfilling Eq. (5). In the transition, reward, and sensor function, the CRV inputs are replaced by the action and observation sets, with a histogram h mapping to ρ being replaced by a set of mappings over all permutations of the values that agents can take according to the counts in h , all mapping to ρ , thereby leading to the functions fulfilling Eq. (7).

B.2 Full Proof for Theorem 2

We repeat Thm. 2 and then provide its proof.

Theorem 2. *A ground DecPOMDP M that meets Eqs. (5) and (7) is equivalent to a counting DecPOMDP $\bar{M}$.*

Proof. First, we convert a DecPOMDP $M = (\mathbf{I}, S, \mathbf{A}, \mathbf{O}, T, R, \bar{\Omega}, \tau)$, in which Eqs. (5) and (7) hold, into a counting DecPOMDP $\bar{M} = (\bar{\mathbf{I}}, \bar{S}, \bar{\mathbf{A}}, \bar{\mathbf{O}}, \bar{T}, \bar{R}, \bar{\Omega}, \tau)$. Agents that share the same action and observation spaces, meaning $A_i = A_j$ and $O_i = O_j$ for $i, j \in \mathbf{I}$ by Eq. (5), as well as exhibit symmetric behaviour as in Eq. (6) are grouped into a partition, storing for each partition a decision variable $A_k = A_i = A_j$ and an observation variable $O_k = O_i = O_j$. To construct the counted functions $\bar{T}$, $\bar{R}$, and $\bar{\Omega}$ based on Eq. (7), we replace $(A_i)_{i \in \mathcal{J}_k}$ with a CRV $\#_{\mathcal{J}_k}[A_k]$ in the inputs of the transition, reward, and sensor function for each partition. We do the same with the observation variables in the sensor function. For each function and each partition, we then construct from a mapping with a partition action $\mathbf{a}_k$ (partition observation $\mathbf{o}_k$) mapping to an outcome ρ a new mapping with a histogram h_k replacing $\mathbf{a}_k$ ($\mathbf{o}_k$), counting how often each $a \in \text{ran}(A_k)$ ($o \in \text{ran}(O_k)$) occurs in $\mathbf{a}_k$ ($\mathbf{o}_k$) for the counts in h_k , and discard all mappings with permutations of $\mathbf{a}_k$ ($\mathbf{o}_k$). Second, we convert a counting DecPOMDP into a DecPOMDP M that meets Eqs. (5) and (7) by setting $\mathbf{I} = \bigcup_k \mathcal{J}_k$, $A_i = A_k$ and $O_i = O_k$ for all $i \in \mathcal{J}_k$ in each partition $\mathcal{J}_k$, making M fulfil Eq. (5), and then essentially expanding the counted functions into their ground versions, i.e., replacing each CRV $\#_{\mathcal{J}_k}[V_k]$, $V \in \{A, O\}$, with a set of variables $V_{k,1}, \dots, V_{k,n_k}$ as inputs and mapping the different inputs $\mathbf{v}$ to an outcome ρ , whenever the histogram representation of $\mathbf{v}$ maps to ρ , which makes M meet Eq. (7).

B.3 Full Proof of Theorem 3

We repeat Thm. 3 and then provide its proof.

Theorem 3. *For a fixed number of partitions K , fixed action and observation spaces, and a fixed horizon τ , the model complexity of a counting DecPOMDP $\bar{M}_c$ is polynomial in N .*

Proof. The histogram space of a CRV for partition $\mathcal{J}_k, k \in \{1, \dots, K\}$ of size n_k with m possible values has size $\binom{n_k+m-1}{m-1} \leq n_k^m$ [13]. As we assume K is fixed and $K \ll N$, n_k and N have the same order of magnitude, i.e., $n_k \leq N$. As such, due to the range sizes of their inputs, the sizes of the functions $\bar{T}$, $\bar{R}$ and $\bar{\Omega}$ lie in $O(s^2 N^{Ka})$, $O(s N^{Ka})$, and $O(s N^{Ko})$, respectively, with $s = |\text{ran}(\bar{S})|$, $a = \max_k |\text{ran}(A_k)|$, and $o = \max_k |\text{ran}(O_k)|$, depending polynomially on N .

B.4 Full Proof of Theorem 4

We repeat Thm. 4 and then provide its proof.

Theorem 4. *The maximum expected utility for a ground DecPOMDP M that meets Eqs. (5) and (7) is equal to the maximum expected utility of the corresponding policy-counted DecPOMDP $\bar{M}$. That is, $\text{MEU}(M) = \text{MEU}(\bar{M})$.*

Proof. Let Π_M be set of possible joint policies in the ground model M . In the policy-counted model $\bar{M}$, we convert any ground joint policy $\pi = (\pi_1, \dots, \pi_N)$ into a joint counted policy of histograms $\mathbf{h}^\pi = (h_1, \dots, h_K)$, where each h_k counts the number of agents in partition $\mathcal{I}_k$ assigned to each representative policy in Π_k . To establish value equivalence, consider any ground policy π and its corresponding policy histogram $\mathbf{h}^\pi$ and an empty joint (counted observation) at the start. Based on Thm. 2, the reward, transition, and observation functions return the same values between the two models given equivalent inputs. As a result, the immediate rewards in Eqs. (2) and (10) are identical, i.e., $R(s, \mathbf{a}) = \bar{R}(s, \mathbf{h}^a)$, with the joint action in Eq. (2) and the joint counted action in Eq. (10) being equivalent, since the same policies are followed between the ground and the policy-counted version and thus, the actions in the roots are identical, adding up to the counts in the counted joint action. The following sum over states is identical in both equations with the transition functions returning the same values. The sum over joint (counted) observations is equivalent, since every ground joint observation can be translated into a joint counted observation, with the counted version combining those ground joint observations with the same counted representation into one summand and multiplying that summand with the number of instances using the multinomial coefficient [23]. The summand value is identical, since the sensor functions return the same value for equivalent inputs. As such, the calculations at the end of the first step are equivalent.

Subsequent utility calculations have as the observation history an equivalent input as just argued. Thus, with the same arguments as before, the further joint (counted) actions are equivalent, with the action histogram being a counted representation of the ground joint action, which also holds for further joint (counted) observations. Therefore, the expected utilities $U_M(\pi)$ and $U_{\bar{M}}(\mathbf{h}^\pi)$ are identical. As this can be done for every ground policy with its corresponding policy histogram, the counted policy space $\mathbf{H}^\pi$ and the ground policy space Π_M yield the same maximum expected utility, that is $MEU(M) = MEU(\bar{M})$.

B.5 Full Proof of Theorem 5

We repeat Thm. 5 and then provide its proof.

Theorem 5. *For a fixed number of partitions K , fixed action and observation spaces, and a fixed horizon τ , the cost of evaluating a joint counted policy and the size of the overall joint counted policy space in a policy-counted DecPOMDP depend polynomially on the number of agents N .*

Proof. For the cost of evaluating a joint counted policy, consider Eq. (10) with a sum over states, which is of size $s = |\text{ran}(S)|$, and a sum over joint counted observations. With $a = \max_k |\text{ran}(A_k)|$, $o = \max_k |\text{ran}(O_k)|$, the space of joint counted observations is structured for each partition by the number of representative policies p and the number of representative observation histories h , which

are given by [16]

$$\begin{aligned} p &= |\Pi_k| = a^{\frac{o^\tau - 1}{o - 1}} \\ h &= |P_k^{0:\tau, o}| = o^\tau \end{aligned}$$

Each representative history can have up to $n_k \leq N$ agents following that particular history with $K \ll N$, which are distributed onto o possible observations. Thus, the number of histograms per partition is given by

$$H = \binom{N + o - 1}{o - 1} \leq N^o$$

Therefore, the size of the joint counted observation space is given by the size of the cross product of K partitions, p representative policies, and h representative histories, i.e., H^{Kph} , which is capped by

$$H^{Kph} \leq (N^o)^{Ka^{o^\tau} o^\tau} = N^{Ka o^{2\tau+1}}$$

and as such, depends polynomially on N . The size of the joint counting policy space is capped by

$$\prod_{k=1}^K \binom{n_k + p - 1}{p - 1} \leq N^{Kp} = N^{Ka \frac{o^\tau - 1}{o - 1}}$$

and thereby, also depends polynomially on N .

B.6 Full Proof of Theorem 6

We repeat Thm. 6 and then provide its proof.

Theorem 6. *Using policy-counted dynamic programming on a policy-counted DecPOMDP $\bar{M}$ is equivalent to using standard dynamic programming on a ground DecPOMDP M , in which Eqs. (5) and (7) hold.*

Proof. We show equivalence by considering the different steps of the dynamic programming operator.

Exhaustive backup: Given Eq. (5), the policies for each agent in a partition $\mathcal{J}_k$ are identical, meaning that the exhaustive backup can be performed using representative policies for each partition.

Value computation: Given that $MEU(\bar{M}) = MEU(M)$ by Thm. 4 and as such $U_M^{\mathbf{h}^n} = U_M^{\boldsymbol{\pi}}$ for equivalent policies $\mathbf{h}^n$ and $\boldsymbol{\pi}$, $\bar{V}'_k = V'_i$ for all $i \in \mathcal{J}_k$ as the value computation is a rewriting of the utility computation, meaning that the value calculations can be performed for counted partition policies at each partition.

Pruning: Since $\bar{V}'_k = V'_i$ for all $i \in \mathcal{J}_k$, Eq. (4) for pruning is identical for all $i \in \mathcal{J}_k$ given a representative policy $\pi_k \in \Pi_k$, meaning that π_k can be checked for pruning once per partition and thus, pruning can be performed for representative policies per partition.

Given the equivalence of the three steps, we can conclude that the overall procedure yields an equivalent result.

B.7 Full Proof of Theorem 7

We repeat Thm. 7 and then provide its proof.

Theorem 7. *For a fixed number of partitions K , fixed action and observation spaces, and a fixed horizon τ , the runtime of policy-counted dynamic programming in a policy-counted DecPOMDP $\bar{M}$ depends polynomially on the number of agents N .*

Proof. Again, we consider the three steps of the dynamic programming operator to show polynomial dependence on the number of agents N , which reuses definitions of Thm. 3.

Exhaustive backup: With $a = \max_k |\text{ran}(A_k)|$, $o = \max_k |\text{ran}(O_k)|$, the number of policies to generate per partition is given by $p = a^{\frac{o^\tau - 1}{o - 1}}$ [16], meaning Kp over all partitions, which is independent of N .

Value calculation: Eq. (13) depends polynomially on N since the sum over joint counted observations depends polynomially on N . In contrast to the recursive calculation of $U_M^{\mathbf{h}^n}$, we do not need to consider representative observation histories for $t > 0$, meaning that the size of the space of histograms is capped by $(No)^{Kp} = N^{\frac{Koa(o^\tau - 1)}{o - 1}}$, which is polynomial in N .

Pruning: Solving the pruning part can be done using a linear programme, which can be solved in time polynomial w.r.t. the number of variables and constraints [25], whose numbers again depends polynomially on N , since the number of variables and constraints depends on the counted policy space $\mathbf{H}^H$, which depends polynomially on N .

Given that each step depends polynomially on N , the overall runtime depends polynomially on N .

C DecTiger Example

We provide the different model definitions, look at model and worst-case policy space sizes, and consider the differences when applying the dynamic programming operator.

C.1 Model Definition

We use the specification of the DecTiger benchmark from the MADP tool box.

¹. Listing 1 shows the original DecTiger version in the MADP input format. The DecPOMDP model reads as follows:

- $\mathbf{I} = \{\text{agent}_1, \text{agent}_2\}$,
- $S, \text{ran}(S) = \{\text{tiger-left}, \text{tiger-right}\} = \{tl, tr\}$,
- $\mathbf{A} = \{A_i\}_{i \in \mathbf{I}}, \forall i \in \mathbf{I} : \text{ran}(A_i) = \{\text{listen}, \text{open-left}, \text{open-right}\} = \{li, ol, or\}$,
- and

¹ <https://github.com/MADPToolbox/MADP/blob/master/problems/dectiger.dpomdp>

$$- \mathbf{O} = \{O_i\}_{i \in \mathbf{I}}, \forall i \in \mathbf{I} : \text{ran}(O_i) = \{\text{hear-left}, \text{hear-right}\} = \{hl, hr\}.$$

with T , R , and Ω ($= \mathbf{O}$ in Listing 1) in tabular notation in full below (lines are reordered compared to the listing to match the order of the inputs in the definitions).

S	A_1	A_2	T_{tl}	T_{tr}	S	A_1	A_2	R	S	A_1	A_2	$\Omega_{hl,hl}$	$\Omega_{hr,hl}$	$\Omega_{hl,hr}$	$\Omega_{hr,hr}$
tl	li	li	1	0	tl	li	li	-2	tl	li	li	0.7225	0.1275	0.1275	0.0225
tl	li	ol	0.5	0.5	tl	li	ol	-101	tl	li	ol	0.25	0.25	0.25	0.25
tl	li	or	0.5	0.5	tl	li	or	9	tl	li	or	0.25	0.25	0.25	0.25
tl	ol	li	0.5	0.5	tl	ol	li	-101	tl	ol	li	0.25	0.25	0.25	0.25
tl	ol	ol	0.5	0.5	tl	ol	ol	-50	tl	ol	ol	0.25	0.25	0.25	0.25
tl	ol	or	0.5	0.5	tl	ol	or	-100	tl	ol	or	0.25	0.25	0.25	0.25
tl	or	li	0.5	0.5	tl	or	li	9	tl	or	li	0.25	0.25	0.25	0.25
tl	or	ol	0.5	0.5	tl	or	ol	-100	tl	or	ol	0.25	0.25	0.25	0.25
tl	or	or	0.5	0.5	tl	or	or	20	tl	or	or	0.25	0.25	0.25	0.25
tr	li	li	0	1	tr	li	li	-2	tr	li	li	0.7225	0.1275	0.1275	0.0225
tr	li	ol	0.5	0.5	tr	li	ol	9	tr	li	ol	0.25	0.25	0.25	0.25
tr	li	or	0.5	0.5	tr	li	or	-101	tr	li	or	0.25	0.25	0.25	0.25
tr	ol	li	0.5	0.5	tr	ol	li	9	tr	ol	li	0.25	0.25	0.25	0.25
tr	ol	ol	0.5	0.5	tr	ol	ol	20	tr	ol	ol	0.25	0.25	0.25	0.25
tr	ol	or	0.5	0.5	tr	ol	or	-100	tr	ol	or	0.25	0.25	0.25	0.25
tr	or	li	0.5	0.5	tr	or	li	-101	tr	or	li	0.25	0.25	0.25	0.25
tr	or	ol	0.5	0.5	tr	or	ol	-100	tr	or	ol	0.25	0.25	0.25	0.25
tr	or	or	0.5	0.5	tr	or	or	-50	tr	or	or	0.25	0.25	0.25	0.25

with $T_{s'} = T(s', s, a_1, a_2)$, $s' \in \{tl, tr\}$ and $\Omega_{o_1, o_2} = \Omega((o_1, o_2), s, a_1, a_2)$, $o_1, o_2 \in \{hl, hr\}$. The transition function only states that as long as both agents only listen, the state does not change (identity). When at least one agent opens a door, the game basically restarts with the new state being set according to a uniform distribution. It is basically a way of keeping the game infinite by resetting the state to an arbitrary one whenever the agents end the game by opening a door (to either lose—tiger, or win—gold). One could argue that opening a door only ends the game and might not necessarily imply a restart. In that case, one would keep the state as is in all cases ((1, 0) distribution for all tl lines, (0, 1) distribution for all tr lines) and re-spawn the game with an arbitrary starting state, sampled from a (0.5, 0.5) distribution, for do-overs.

The DecTiger model has the same action and observation sets for both agents and exhibits a counting symmetry. Thus, it can be rewritten into a counting model (Def. 3) with $K = 1$. We index the one partition with c .

- $\bar{\mathbf{I}} = \mathfrak{I}_c = \{\text{agent}_1, \text{agent}_2\}$,
- $S = \{tl, tr\}$,
- $\bar{A}_c = \{\#\mathfrak{I}_c[A_c]\}$, where the local action range is $\text{ran}(A_c) = \text{ran}(A_i) = \{li, ol, or\}$, A_i from the ground DecPOMDP definition,
- $\bar{T}(S', S, \bar{A}_c) = P(S' | S, \bar{A}_c)$,

- $\bar{R}(S, \bar{A}_c)$,
- $\bar{O}_c = \{\#_{\mathcal{I}_k}[O_k]\}$, where the local observation range is $\text{ran}(O) = \{hl, hr\}$.
- $\bar{\Omega}(\bar{O}_c, S) = P(\bar{O}_c | S)$

with $\bar{T}$, $\bar{R}$, and $\bar{\Omega}$ following after the specification of the policy-counted DecPOOMDP version.

In a policy-counted DecPOMDP (Def. 5), the action and observation variables are plain variables again. The CRVs only occur within the transition, reward, and sensor functions. Specifically, for the policy-counted DecTiger example, the model is defined as follows:

- $\bar{\mathcal{I}} = \mathcal{I}_c = \{agent_1, agent_2\}$,
- $S = \{tl, tr\}$,
- $\bar{A} = \{A_c\}$, $\text{ran}(A_c) = \text{ran}(A_i) = \{li, ol, or\}$,
- $\bar{T}(S', S, \bar{A}_c) = P(S' | S, \bar{A}_c)$,
- $\bar{R}(S, \bar{A}_c)$,
- $\bar{O} = \{O_k\}_{k=1}^K$, $\text{ran}(O_c) = \{hl, hr\}$.
- $\bar{\Omega}(\bar{O}_c, S) = P(\bar{O}_c | S)$

with $\bar{A}_c$ and $\bar{O}_c$ defined as for the counting DecPOMDP above.

The functions $\bar{T}$, $\bar{R}$, and $\bar{\Omega}$ are given below in tabular notation with $\#A$ short for $\#_{\mathcal{I}_k}[A_k]$ (histogram positions: $[li, ol, or]$) and $\bar{\Omega}_h$ for $\bar{\Omega}(S, \#A, \#_{\mathcal{I}_k}[O_k] = h)$:

S	$\#A$	$\bar{T}_{tl}$	$\bar{T}_{tr}$	S	$\#A$	$\bar{R}$	S	$\#A$	$\bar{\Omega}_{[2,0]}$	$\bar{\Omega}_{[1,1]}$	$\bar{\Omega}_{[0,2]}$
tl	$[2, 0, 0]$	1.0	0.0	tl	$[2, 0, 0]$	−2	tl	$[2, 0, 0]$	0.7225	0.1275	0.0225
tl	$[1, 1, 0]$	0.5	0.5	tl	$[1, 1, 0]$	−101	tl	$[1, 1, 0]$	0.25	0.25	0.25
tl	$[1, 0, 1]$	0.5	0.5	tl	$[0, 2, 0]$	−50	tl	$[0, 2, 0]$	0.25	0.25	0.25
tl	$[0, 1, 1]$	0.5	0.5	tl	$[0, 1, 1]$	−100	tl	$[0, 1, 1]$	0.25	0.25	0.25
tl	$[0, 2, 0]$	0.5	0.5	tl	$[0, 0, 2]$	20	tl	$[0, 0, 2]$	0.25	0.25	0.25
tl	$[0, 0, 2]$	0.5	0.5	tl	$[1, 0, 1]$	9	tl	$[1, 0, 1]$	0.25	0.25	0.25
tr	$[2, 0, 0]$	0.0	1.0	tr	$[2, 0, 0]$	−2	tr	$[2, 0, 0]$	0.7225	0.1275	0.0225
tr	$[1, 1, 0]$	0.5	0.5	tr	$[1, 1, 0]$	9	tr	$[1, 1, 0]$	0.25	0.25	0.25
tr	$[1, 0, 1]$	0.5	0.5	tr	$[0, 2, 0]$	20	tr	$[1, 0, 1]$	0.25	0.25	0.25
tr	$[0, 1, 1]$	0.5	0.5	tr	$[0, 1, 1]$	−100	tr	$[0, 2, 0]$	0.25	0.25	0.25
tr	$[0, 2, 0]$	0.5	0.5	tr	$[0, 0, 2]$	−50	tr	$[0, 1, 1]$	0.25	0.25	0.25
tr	$[0, 0, 2]$	0.5	0.5	tr	$[1, 0, 1]$	−101	tr	$[0, 0, 2]$	0.25	0.25	0.25

Note that the rows do not add up to one because the histogram $[1, 1]$ in $\bar{\Omega}$ stands for two joint observations, (hl, hr) and (hr, hl) , meaning that the probability for $\bar{\Omega}_{[1,1]}$ has to be counted twice for the probability distribution to add up to 1. In general, a multinomial coefficient provides the number of inputs represented by a histogram, i.e., $n_k! / \prod_l n_l!$.

Before moving on to model and policy space sizes, we list the DecTiger specification in the MADP toolbox.

Listing 1 DecTiger specification in the MADP toolbox (without the comments from the source)

```

agents: 2
discount: 1
values: reward
states: tiger-left tiger-right
start: uniform
actions:
listen open-left open-right
listen open-left open-right
observations:
hear-left hear-right
hear-left hear-right
# Transition probabilities
T: * : uniform
T: listen listen : identity
# Observation probabilities: <2 actions> : <state> : <2
  observations> : probability
O: * : uniform
O: listen listen : tiger-left : hear-left hear-left : 0.7225
O: listen listen : tiger-left : hear-left hear-right : 0.1275
O: listen listen : tiger-left : hear-right hear-left : 0.1275
O: listen listen : tiger-left : hear-right hear-right : 0.0225
O: listen listen : tiger-right : hear-left hear-left : 0.7225
O: listen listen : tiger-right : hear-left hear-right : 0.1275
O: listen listen : tiger-right : hear-right hear-left : 0.1275
O: listen listen : tiger-right : hear-right hear-right :
  0.0225
# Rewards: <2 actions> : <state> : * : * : reward
R: listen listen: * : * : * : -2
R: open-left open-left : tiger-left : * : * : -50
R: open-right open-right : tiger-right : * : * : -50
R: open-left open-left : tiger-right : * : * : 20
R: open-right open-right : tiger-left : * : * : 20
R: open-left open-right : tiger-left : * : * : -100
R: open-left open-right : tiger-right : * : * : -100
R: open-right open-left : tiger-left : * : * : -100
R: open-right open-left : tiger-right : * : * : -100
R: open-left listen : tiger-left : * : * : -101
R: listen open-right : tiger-right : * : * : -101
R: listen open-left : tiger-left : * : * : -101
R: open-right listen : tiger-right : * : * : -101
R: listen open-right : tiger-left : * : * : 9
R: listen open-left : tiger-right : * : * : 9
R: open-right listen : tiger-left : * : * : 9
R: open-left listen : tiger-right : * : * : 9

```

C.2 Model and Policy Space Sizes

The following table characterises the ground, counting, and policy-counted model M , $\bar{M}_c$, $\bar{M}$ regarding different parameters:

- Number of agents: N , number of partitions: K , horizon τ
- State size: s
- Size of a local / partition action: a , that is $|\text{ran}(A_i)|$, $|\text{ran}(\#_{\mathcal{I}}[A_c])|$, $|\text{ran}(A_c)|$, respectively
- Size of the joint (counted) action: $\mathbf{a}$
- Size of a local / partition observation: o , that is $|\text{ran}(O_i)|$, $|\text{ran}(\#_{\mathcal{I}}[O_c])|$, $|\text{ran}(O_c)|$, respectively
- Size of the joint (counted) action: $\mathbf{o}$
- Size of local / partition policy space per state: p , i.e., local policy π over A_i, O_i , partition policy over $\#_{\mathcal{I}}[A_c], \#_{\mathcal{I}}[O_c]$, counted policy over representative policies over A_k, O_k
- Size of the joint (counted) policy space per state: $\mathbf{p}$
- Size of the transition, reward, and sensor functions by range sizes of the inputs: T, R, Ω

	M	$\bar{M}_c$	$\bar{M}$
s	2	2	2
a	3	$\binom{N+3-1}{3-1} = \binom{N+2}{2}$	3
$\mathbf{a}$	3^N	$\binom{N+2}{2}^K$	$\binom{N+2}{2}^K$
o	2	$\binom{N+2-1}{2-1} = \binom{N+1}{1}$	2
$\mathbf{o}$	2^N	$\binom{N+1}{1}^K$	$\binom{N+1}{1}^K$
p	$3^{\frac{2^\tau-1}{2-1}}$	$\binom{N+2}{2}^{\frac{\binom{N+1}{1}^{\tau-1}}{\binom{N+1}{1}-1}}$	$\left(N + 3^{\frac{2^\tau}{2-1}} - 1\right)^{\frac{2^\tau}{2-1}}$
$\mathbf{p}$	$3^{N \frac{2^\tau-1}{2-1}}$	$\binom{N+2}{2}^K \frac{\binom{N+1}{1}^{\tau-1}}{\binom{N+1}{1}-1}$	$\left(N + 3^{\frac{2^\tau}{2-1}} - 1\right)^K$
T	$2 \cdot 2 \cdot 3^N$	$2 \cdot 2 \cdot \binom{N+2}{2}^K$	$2 \cdot 2 \cdot \binom{N+2}{2}^K$
R	$2 \cdot 3^N$	$2 \cdot \binom{N+2}{2}^K$	$2 \cdot \binom{N+2}{2}^K$
Ω	$2 \cdot 2^N$	$2 \cdot \binom{N+1}{1}^K$	$2 \cdot \binom{N+1}{1}^K$

C.3 Dynamic Programming

We consider the three steps of dynamic programming (backup, value calculation, pruning) for the levels $t = 0$ and $t = 1$ for the three versions of the DecTiger example.

Level $t = 0$

Backup: The operator for the ground version builds *three* policies for each of its two agents, each policy having a single (root) node for one of the actions *li, ol, or*, building six policies overall. A counting programming operator for the counting version would build *six* policies for its one partition, each with a single (root) node for one of the action histograms $[2, 0, 0]$, $[1, 1, 0]$, $[1, 0, 1]$, $[0, 1, 1]$, $[0, 2, 0]$, $[0, 0, 2]$. The counted programming operator for the policy-counted version builds *three* representative policies for its *one* partition, each with a single (root) node for one of the actions *li, ol, or*. With two available agents, there are six counted policies over the three representative ones: $[2, 0, 0]$, $[1, 1, 0]$, $[1, 0, 1]$, $[0, 1, 1]$, $[0, 2, 0]$, $[0, 0, 2]$.

Value Computation: For the ground version, the operator calculates the values for each of its *two* agents for each of its three policies. For each policy, the computation has to be performed for each state tl, tr and each of the other agent's three policies. For the counting version, the operator calculates the values for its *one* partition for each of its *six* policies. For each policy, the computation has to be performed for each state tl, tr . For the policy-counted version, the operator calculates the values for its *one* partition for each of its *six* counted policies. For each policy, the computation has to be performed for each state tl, tr .

Pruning: In the ground version, the operator checks for each of the *two* agents, whether one of its three policies is (weakly) dominated by one of the other two policies for each state tl, tr and each of the three policies of the other agent (does not need to be the same policy over all states and policies of the other agent), until there are no more policies to prune. In the counting version, the operator would check for its *one* partition, whether any of its six policies is (weakly) dominated by one of the other five policies for each state tl, tr , until there are no more policies to prune. In the policy-counted version, the operator checks for its *one* partition, whether any of its three representative policies is dominated by one of the other two policies for each state tl, tr and each of the other counted policies over the remaining partition, which contains only one agent, i.e., $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$ (that is just the three representative policies), until there are no more policies to prune.

Comment: The difference between the counting and policy-counted versions is not as pronounced at this level, with the ground version also working within the same order of magnitude.

Level $t = 1$

Backup: With no policies pruned, the operator for the ground version builds for its *two* agent each 27 policies out of its previous three policies and two possible observations hl, hr , with each policy having a root node and two children, one for each observation. The actions in the nodes are each possible combination of li, ol, or . With no policies pruned, the operator for the counting version would build for its *one* partition 216 policies out of its previous six policies and three possible counted observations $[0, 2], [1, 1], [2, 0]$. The policies would have a root node and three children, one for each possible histogram, with the nodes containing each possible combination of the six action histograms. The operator for the policy-counted version builds the same 27 (representative) policies out of its previous three representative policies and two possible observations hl, hr that the ground version has. With two available agents, there are then 378 counted policies.

Value Computation: For the ground version, the operator performs the same calculations outlined for level $t = 0$ for its 27 policies for each combination of state tl, tr and policy of the other agent (27 as well) for each agent. For the counting version, the operator calculates the values for its one partition for each of its 216 policies for each state tl, tr . For the policy-counted version, the operator calculates the values for its one partition for each of the 378 counted policies. For each policy, the computation has to be performed for each state tl, tr .

Pruning: Pruning again follows the same procedure as outlined above, for two agents and each of their 27 policies in each state and for each of the other agent's policies in the ground version. In the counting version, the 1296 policies of its one partition get checked for each state tl, tr . In the policy-counted version, the 27 representative policies of its one partition are checked for each state and each counted policy of the remaining partition, i.e., the 27 representative policies.

Comment: This numerical example illustrates impressively that policy counting only pays off if the number of agents is large or at least outweighs the number of representative policies. That is the number of agents should at least be an order larger than the number of representative actions and observations available.